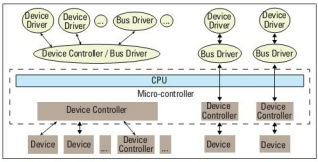

Figure 1: Device and driver interaction

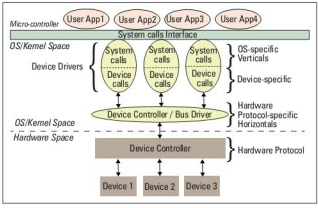

Figure 2: Linux device driver partition

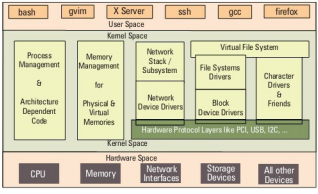

Figure 3: Linux kernel overview

areas is mainly a Linux porting effort, which is typically done for a new CPU or architecture. Moreover, the code in these two verticals cannot be loaded or unloaded on the fly, unlike the other three verticals. Henceforth, when we talk about Linux device drivers, we mean to talk only about the latter three verticals in Figure 3.

Let's get a little deeper into these three verticals. The network vertical consists of two parts: a) the network protocol stack, and b)the network interface card (NIC) device drivers, or simply network device drivers, which could be for Ethernet, Wi-Fi, or any other network horizontals. Storage, again, consists of two parts: a) File-system drivers, to decode the various formats on different partitions, and b) Block device drivers for various storage (hardware) protocols, i.e., horizontals like IDE, SCSI, MTD, etc.

With this, you may wonder if that is the only set of devices for which you need drivers (or for which Linux has drivers). Hold on a moment; you certainly need drivers for the whole lot of devices that interface with the system, and Linux does have drivers for them. However, their byte-oriented cessibility puts all of them under the character vertical—this is, in reality, the majority bucket. In fact, because of the vast number of drivers in this vertical, character drivers have been further sub-classified—so you have *tty* drivers, input drivers, console drivers, frame-buffer drivers, sound drivers, etc. The typical horizontals here would be RS232, PS/2, VGA, I^2C, I^2S, SPI, etc.

Multiple-vertical drivers

One final note on the complete picture (placement of all the drivers in the Linux driver ecosystem): the horizontals like USB, PCI, etc, span below multiple verticals. Why is that? Simple—you already know that you can have a USB Wi-Fi dongle, a USB pen drive, and a USB-to-serial converter—all are USB, but come under three different verticals!

In Linux, bus drivers or the horizontals, are often split into two parts, or even two drivers: a) device controller-specific, and b) an abstraction layer over that for the verticals to interface, commonly called cores. A classic example would be the USB controller drivers *ohci*, *ehci*, etc, and the USB abstraction, *usbcore*.

Summing up

So, to conclude, a device driver is a piece of software that drives a device, though there are so many classifications. In case it drives only another piece of software, we call it just a driver. Examples are file-system drivers, *usbcore*, etc. Hence, all device drivers are drivers, but all drivers are not device drivers.

"Hey, Pugs, hold on; we're getting late for class, and you know what kind of trouble we can get into. Let's continue from here, tomorrow," exclaimed Shweta. Jumping up, Pugs finished his explanation: "Okay. This is the basic theory about device drivers. If you're interested, later, I can show you the code, and all that we have been doing for the various kinds of drivers." And they hurried towards their classroom. END

By: Anil Kumar Pugalia

The author is a freelance trainer in Linux Internals, Linux Device Drivers, Embedded Linux & related topics. Prior to this he had been in Intel & Nvidia. He has been working with Linux since 1994. gold medallist from the Indian Institute of Science, Linux & knowledge sharing are two of his many passions. More can be found at http://profession.sarika-pugs.com. He can be reached at email@sarika-pugs.com